

Good — here's the updated `Program.cs` with a chat loop that keeps running until you type `exit`, and also keeps track of conversation history so follow-up questions make sense.

```
```csharp
```

```
class Program
```

```
{
```

```
 static async Task Main()
```

```
 {
```

```
 var embedder = new EmbeddingService();
```

```
 var store = new VectorStore();
```

```
 var chat = new ChatService();
```

```
 // Load and index your document (only done once at startup)
```

```
 Console.WriteLine("Loading and indexing document...");
```

```
 string documentText = File.ReadAllText("mydocument.txt");
```

```
 var chunks = DocumentProcessor.ChunkText(documentText);
```

```
 foreach (var chunk in chunks)
```

```
 {
```

```
 var vector = await embedder.GetEmbedding(chunk);
```

```
 store.Add(chunk, vector);
```

```
 }
```

```
 Console.WriteLine($"Indexed {chunks.Count} chunks. Ready to chat!\n");
```

```
 // Keep conversation history across turns
```

```
 var conversationHistory = new List<(string role, string content)>();
```

```
while (true)
{
 Console.Write("You: ");
 string question = Console.ReadLine();

 if (string.IsNullOrEmpty(question))
 continue;

 if (question.Trim().ToLower() == "exit")
 {
 Console.WriteLine("Goodbye!");
 break;
 }

 // Find relevant chunks for this question
 var queryVector = await embedder.GetEmbedding(question);
 var relevantChunks = store.Search(queryVector);

 // Ask Llama, passing both document context and chat history
 string answer = await chat.Ask(question, relevantChunks, conversationHistory);

 Console.WriteLine($"Bot: {answer}\n");

 // Save this turn to history
 conversationHistory.Add(("user", question));
```

```

 conversationHistory.Add("assistant", answer));
 }
}
...

```

Update `ChatService.Ask` to accept and use the conversation history:

```

` `` `csharp
class ChatService
{
 private readonly HttpClient _http = new();

 public async Task<string> Ask(string question, List<string> contextChunks, List<(string
role, string content)> history)
 {
 var context = string.Join("\n---\n", contextChunks);

 var systemPrompt = $"Answer ONLY using the following document context. " +
 $"If the answer isn't in the context, say you don't
know.\n\nContext:\n{context}";

 // Build message list: system prompt + prior turns + new question
 var messages = new List<object>
 {
 new { role = "system", content = systemPrompt }
 };
 }
}

```

```

foreach (var (role, content) in history)
{
 messages.Add(new { role, content });
}

messages.Add(new { role = "user", content = question });

var response = await _http.PostAsJsonAsync("http://localhost:11434/api/chat", new
{
 model = "llama3.1",
 messages,
 stream = false
});

var result = await response.Content.ReadFromJsonAsync<ChatResponse>();
return result.message.content;
}
}
...

```

## ## What changed

- The document is indexed **once** at startup, not on every question
- A `while(true)` loop keeps asking for input until you type `exit`
- `conversationHistory` is passed to `ChatService.Ask` each time, so the bot remembers earlier turns (e.g. "what about the second point?" will make sense)

- Each new question still does a fresh retrieval search against your document, so it stays grounded in the source content

## Try it

...

You: What is the refund policy?

Bot: [answer based on document]

You: What about for digital products specifically?

Bot: [answer, understanding "what about" refers to the refund policy]

You: exit

Goodbye!

...

One thing to watch: as history grows, you're sending more tokens each turn, which slows things down and can eventually exceed context length. Want me to add a simple cap (e.g., keep only the last 5-10 turns) so it stays fast in longer sessions?